

Unity Piano-Tiles Style Rhythm Game – Deep Research Digest

Executive Summary

Building a piano-tiles-style rhythm game involves balancing audio quality, performance, and practicality. **Sample selection** is critical: using a moderate set of high-quality piano note samples (e.g. 30–70 notes) and filling gaps via pitch-shifting can drastically reduce memory while maintaining fidelity ¹ ². On mobile, compressed formats like Ogg Vorbis provide good size-to-quality ratios, whereas ADPCM offers faster decoding with a larger footprint ³. **Real-time audio architecture** should prioritize low latency and efficient note triggering. In Unity, this means using an **AudioSource pool** sized for peak polyphony, setting the DSP buffer to **Best Latency**, and preloading or decompressing clips to avoid runtime hiccups ⁴. Unity's built-in audio engine (built atop FMOD) is sufficient for a first release, offering simplicity and 60 FPS synchronization, while middleware like FMOD Studio or Wwise can be integrated later for advanced scheduling and mixing features ⁵ ⁶.

A successful repertoire strategy will cover the **full classical piano range** (typical pieces rarely exceed the 88-key span ⁷) and leverage the public-domain status of composers like Bach, Mozart, Chopin for content. All these compositions are free to use, but developers must ensure the specific **MIDI files or scores** have no additional copyrights ⁸ ⁹. Libraries like **DryWetMIDI** (for MIDI) and **MusicXml.Net** (for MusicXML) can parse scores into game-ready data structures ¹⁰ ¹¹, enabling a data-driven chart pipeline. This pipeline converts tempo maps and note events into a JSON chart format for runtime use, possibly slicing the data into manageable time chunks for smooth streaming.

Performance benchmarks from similar tap-tile games indicate that continuous 60 FPS updates with frequent audio events can strain mid-range phones. Careful pooling and voice management (e.g. capping polyphony around ~40 voices on low-end devices ¹²) will prevent audio dropouts. Techniques like **voice stealing** (dropping/virtualizing low-priority notes when the pool is exhausted) and Unity's built-in voice priority system ¹³ ¹² help handle extreme note bursts (~200 notes/sec). Finally, polish can be added with **enhancements** such as lightweight reverb, velocity-based EQ adjustments, and subtle randomization of pitch/volume per note to avoid machine-gun effect. However, expensive effects (e.g. long convolution reverbs) should be avoided on mobile ¹⁴.

Actionable Unity Steps: Use **Ogg Vorbis** for note samples unless profiling shows a CPU bottleneck (in which case try ADPCM) ³. Stick with Unity's audio engine and an AudioSource pool for now, set **DSP Buffer Size = Best Latency**, and mark note AudioClips as **Decompress on Load** for minimal playback delay ⁴. Ensure all classical content is genuinely public-domain (consider transcribing scores yourself or using reputable PD MIDI sources), and include attributions or disclaimers as needed to satisfy app store policies. Below, we delve into each aspect in detail with supporting research and examples.

1. Sample-Set Strategy (Piano Note Assets)

Optimal sample count: Mobile rhythm games must balance audio fidelity with file size and memory. Rather than using all 88 piano keys sampled, many implementations use a subset of notes and pitch-shift the rest. Using ~30–50 samples (e.g. sampling every 2–3 semitones) can cover the keyboard with

minimal quality loss ¹. For instance, one soundfont uses only 13 distinct piano notes to cover the full range (sampling roughly every 4th key) across 5 velocity layers (65 samples total) ². This approach yields a much smaller footprint than a full 88-sample set. However, the trade-off is audible if stretched too far – shifting more than about **3–4 semitones** from a sample's original pitch can introduce noticeable timbral artifacts (unnaturally chipmunked highs or booming lows). Thus, higher sample counts (70+ or full 88) provide better consistency at the cost of larger size. A commercial Unity piano asset with **full 88-key sampling** is ~166 MB in size ¹⁵, whereas a scaled-down set (e.g. one sample per 3 keys) can be an order of magnitude smaller.

Filling the gaps: For notes without a dedicated sample, real-time **pitch shifting** is the typical solution. A high-quality resampling algorithm can transpose a sample up or down a few semitones with minimal quality loss. It's wise to distribute your chosen sample notes so that no note has to be transposed more than a couple of semitones up or down. In practice, sampling at least one note per minor third (every 3–4 semitones) is a good compromise, keeping transposition within $\sim \pm 2$ semitones. At extreme ends of the piano, you may sample a bit more densely (the lowest and highest octaves are more timbrally sensitive to stretching). If a needed pitch falls outside the recorded range (e.g. if you didn't record the very top note), you can still pitch-shift the highest or lowest sample, but the timbre might suffer noticeably – plan your sample set to include the full range of pitches that your repertoire demands (more on repertoire range below).

Velocity layers: A piano's tone changes with playing intensity, so professional sample libraries include multiple **velocity layers** per note (soft, medium, hard strikes). On mobile, including velocity layers improves authenticity but significantly adds to asset size. Even two layers (pp vs. ff) doubles the number of samples. Decide early if you will support dynamics in gameplay – if the game simply triggers notes at fixed volume, a single layer might suffice. For more expressive playback, consider at least 2–3 layers for important pieces. Some optimized libraries manage this by sampling fewer keys but at many velocities (e.g. the earlier example of 13 keys $\times$ 5 velocities ²). If storage is a concern, an alternative is **single-layer sampling combined with dynamic EQ** (discussed in section 6): use one sample per note and simulate softer vs. harder timbres via filtering. This saves space at the cost of true recorded nuance.

Round-robin alternates: Repetition of the exact same sample can lead to a mechanical “machine-gun” effect, especially for trills or replayed notes. To combat this without recording every note twice, you can implement a **round-robin** system: have two (or more) slightly different recordings for the same note and alternate between them on successive plays. This reduces the ear's ability to detect identical waveforms. If recording multiple takes per note isn't feasible, **micro-randomization** (slight random changes in pitch or volume each time) can approximate a similar effect (more in section 6). Round-robins do increase the sample count (and memory usage) linearly, so budget accordingly (e.g. two takes of 40 samples = 80 files). Many modern sample libraries use round-robin for percussion or short notes; for piano, it's less critical than for drums, but can still help with very fast repeated notes.

Storage and memory footprint: Storing dozens of high-quality audio files on a mobile app requires careful compression choices. Unity supports multiple audio compression formats, each with pros/cons:

- **Ogg Vorbis:** Excellent compression (small file size) at decent quality, and **Unity decodes it natively** on all platforms ¹⁶. Vorbis at default settings can shrink a WAV by ~85-90%. The downside is CPU cost: Vorbis is relatively complex to decode. Unity typically handles this by either *decompressing on load* (using more memory but no runtime decode) or *compressed playback* (decoding on the fly). Vorbis is a good default for many short clips if memory allows full decompression. It keeps app download size low.

- **ADPCM:** A simpler waveform codec (~4:1 compression). Unity's ADPCM is faster to decode than Vorbis and uses less CPU, which can be important if many notes play at once ¹⁷. The trade-off is larger file sizes than OGG for equivalent quality (ADPCM has audible noise at high frequencies and doesn't compress silence as well). In Unity tests, **ADPCM in Compressed-in-Memory mode used less RAM than Vorbis at 100% quality and much lower CPU load** ³. This suggests that if you want to play clips in compressed form (not fully decompressed), ADPCM is a safer bet for performance. ADPCM is a good choice if you need to keep clips compressed in RAM or stream them, but you have some storage headroom to afford the bigger files.
- **AAC/MP3:** These are common music codecs. Unity on iOS/Android can import MP3/AAC and will use hardware decoding for them in some cases. However, **Unity's documentation notes no clear benefit to using MP3 on mobile** – in tests it had *larger file size and higher CPU* than Vorbis or ADPCM, plus known looping bugs ¹⁸. AAC is similar (and not directly exposed as an import option in Unity Editor). Overall, AAC/MP3 aren't recommended inside Unity for short sound effects or notes. Stick to Vorbis or ADPCM, which Unity's audio pipeline is optimized for.
- **PCM (Uncompressed WAV):** Only viable for very short sounds due to size. It has zero decode overhead (just raw data), but an 88-note piano uncompressed would be enormous (tens of MB of memory per layer). You'd only use PCM for tiny blips or if you absolutely cannot afford any decoding latency and have lots of RAM to spare – not the case on most mobiles.

Memory vs CPU trade-offs: Unity allows per-clip **Load Type** settings: **Decompress on Load** (entire clip is loaded as PCM into memory), **Compressed in Memory** (keep it compressed in RAM and decode during playback), or **Streaming** (for long music tracks, stream from disk). For piano notes (short clips), **Decompress on Load** is often ideal for latency – the note is ready in memory, eliminating decode stutter at play time ⁴. The cost is higher RAM usage: all samples will occupy their full PCM size. If you have, say, 50 stereo clips at 3 seconds each, 16-bit 44.1 kHz, that's $\sim 50 \times 0.26 \text{ MB} \approx 13 \text{ MB}$ in memory. That's acceptable on modern phones (and you can unload banks for unused songs), especially considering Unity games often use tens of MB for textures. If memory gets tight, you could switch lesser-used samples to **Compressed in Memory** with ADPCM compression, which trades a bit of CPU to cut memory roughly by 75% ¹⁷. Avoid Vorbis in Compressed-in-Memory at 100% quality – it can actually use more RAM than ADPCM and burn CPU without much gain ³. As a rule, use **Vorbis+DecompressOnLoad** for most clips (small files on disk, moderate RAM), or **ADPCM+CompressedInMemory** if you need to shrink memory and can handle minimal quality loss. **Streaming** is not recommended for hundreds of short notes; it's meant for long music files and might choke if you attempted to stream many files simultaneously ¹⁹.

In summary, aim for a **moderate sample set** (not full 88 unless truly needed) with a **high-quality compression format**. For initial development, using Ogg Vorbis at Unity's default quality and Decompress on Load is a safe, simple approach ¹⁶ ²⁰. If profiling shows audio decoding spikes, you can experiment with ADPCM compression or preloading strategies. This strategy yields a relatively small app size and smooth playback of the note samples on demand.

2. Real-Time Playback Architecture in Unity

Achieving **low-latency, glitch-free note playback** in Unity requires tuning engine settings and managing AudioSource objects carefully. Mobile devices, especially Android phones, are notorious for audio latency, so every millisecond we can save matters in a rhythm game.

Unity audio settings for low latency: In Unity's Project Settings > Audio, set the **DSP Buffer Size** to **Best Latency** ⁴. This configures Unity's audio engine to use a smaller audio buffer, reducing the delay between triggering a sound and hearing it. The default buffer (Best Performance) is larger (often 1024 samples or more) and can add tens of milliseconds of lag. Best Latency typically uses a 256-sample buffer (or lower, platform-dependent) ²¹. This trades some CPU overhead for responsiveness. On iOS, Unity can often achieve sync that feels instant (~20 ms). On Android, even with the smallest buffer, you might see ~50 ms or more latency depending on the device hardware ²². (Android's audio subsystem varies widely—some devices add extra buffering that Unity cannot fully bypass.) Still, setting Best Latency is essential. Be aware that on some low-end devices this could risk occasional audio glitches if the CPU can't fill the small buffer in time (a buffer underrun). Most 2019+ mid-range phones should handle it, especially as our audio clips are short and can be predecoded.

Next, **disable any Unity audio features that add latency**. Do not use **DSP effects** (like reverb filters or complex AudioMixers) on the critical path for note sounds, as they may introduce additional buffering. If you use Unity's AudioMixer, turn off **Automatic DSP sync** for snapshots or etc., and avoid using Time Stretch or Pitch Shifter effects which add delay. The safest route is to play the note samples dry (no effects) and apply any needed effects manually in the clip assets or via lightweight filters. The Medium article by Thibaut Dumont (2017) specifically advises minimizing audio effects for rhythm games ⁴. If you need volume leveling or a bit of EQ, you can apply it in an AudioMixer **with a constant latency of zero** (most simple mixers don't add latency, but check Unity's documentation if you add filters). We'll cover reverb considerations in section 6.

AudioSource pooling: Unity requires an **AudioSource** component to play each sound. Spawning a new AudioSource every time a note is needed would be too slow and would thrash the garbage collector. Instead, create a **pool of AudioSource objects** at startup. For example, you might create an array of, say, 32 AudioSources attached to a single GameObject (or spread across note objects). Reuse these in a round-robin fashion for each note event. That means when a note needs to play, you grab the next available AudioSource from the pool, assign the appropriate AudioClip and pitch, and call `Play()` (or `PlayScheduled()` – more on scheduling shortly). This way, you avoid runtime instantiation costs and ensure that a high burst of notes doesn't exceed a fixed number of sources.

- **Pool size:** Determine this by the maximum polyphony you expect. A complex classical piece with pedal could easily have 10–20 notes ringing concurrently (e.g. a big Chopin chord with sustain). Rapid arpeggios or trills might queue up many short decays at once. Unity's default **voice limit is 32** sounds at once (beyond that, it will start culling voices) ²³. You can increase this in Project Settings > Audio if needed (up to 256) ²⁴, but there's a performance cost to extremely high counts. In practice, keeping **real voices ≈40 or fewer on mobile is recommended** ²⁵. For safety, you might allocate ~50 AudioSources in the pool. This doesn't mean all 50 will play at once (hopefully not), but it prevents missing a note during very dense passages. If the pool does run out, the strategy could be either to **steal** the oldest voice (stop the longest-running note to free a source) or simply drop the new note. Implementing voice stealing yourself gives more control: you could fade out the stolen note gracefully to avoid a pop. However, Unity can handle some of this for you via its **voice priority system**.
- **Voice prioritization:** Each AudioSource in Unity has a Priority value (0 = highest). Unity will automatically cut off (virtualize) the lowest-priority sounds first when the voice limit is exceeded ¹³. You can use this to ensure important sounds (like the main melody or a metronome click) never drop. In our case, all piano notes are similar, so priority might just remain equal for all, and Unity would then cull the least audible (i.e. quietest or farthest) if needed ¹³. Since this is a music game and every note matters, ideally we avoid culling. So it's best to **raise the voice limit** slightly above expected needs (e.g. to 64) while also keeping the pool large enough.

Scheduling vs immediate play: Unity provides an API to schedule audio playback on the DSP clock: `AudioSource.PlayScheduled(time)`. This is extremely useful for tight rhythm synchronization. Instead of calling `Play()` exactly when a note is due (which can vary a few milliseconds depending on frame timing), you can schedule each note a little in advance to land exactly on beat. For example, you could, at song start, schedule all notes at their exact timestamps according to the chart. However, scheduling too far ahead removes interactivity (you can't stop easily or handle misses). A common approach is a rolling scheduler: always schedule, say, 0.1–0.2 seconds ahead of the current time. This ensures jitter-free timing (the audio thread wakes up and starts the clip at the exact DSP timestamp). The good news: if you keep frame rate at 60 FPS and logic updated, simply triggering `Play()` within $\sim \pm 5$ ms of the intended time is often sufficient, especially with Best Latency settings. But if you find note timing isn't tight, consider using `PlayScheduled(AudioSettings.dspTime + offset)` where offset is perhaps 2–3 frames worth of time. This can compensate for the ~ 0.1 s **AudioSource.time** polling delay that was observed in tests ²⁶. Advanced scheduling is more relevant if you have a steady background music and need to align notes perfectly with it. In our design, we have no pre-rendered backing track – we *are* the music – so we have flexibility. Still, micro-timing matters for player feel.

Example note player (pseudocode): Below is a simple example of how to implement a pooled note player with pitch shifting and humanization:

```
public class PianoNotePlayer : MonoBehaviour {
    public AudioSource[] audioPool;
    private int poolIndex = 0;
    // Optional: parameters for subtle randomization
    public float pitchRandomCentRange = 2f;    // ±2 cents
    public float volumeRandomRange = 0.05f;    // ±5%

    void Start() {
        // Assuming audioPool is already filled with AudioSource instances in
        // editor or Awake.
    }

    // Play a piano note with specified clip and base pitch (1.0 = original).
    public void PlayNote(AudioClip clip, float basePitch, float volume =
1.0f) {
        // Grab next AudioSource from pool
        AudioSource src = audioPool[poolIndex];
        poolIndex = (poolIndex + 1) % audioPool.Length;

        // Set clip and properties
        src.clip = clip;
        // Apply a slight random detune and volume variation for realism
        float randCent = Random.Range(-pitchRandomCentRange,
pitchRandomCentRange);
        // Convert cents to pitch multiplier (1 cent = 2^(1/1200))
        src.pitch = basePitch * Mathf.Pow(2f, randCent/1200f);
        src.volume = volume * Random.Range(1f - volumeRandomRange, 1f +
volumeRandomRange);

        src.Play(); // or src.PlayScheduled for precise timing
    }
}
```

```
}
}
```

In this pseudocode, we cycle through the `audioPool` array for each note. We adjust `src.pitch` to include a random detune up to ± 2 cents (which is a very subtle variation – human ear barely notices it individually, but it breaks the perfect uniformity). We also randomize volume by $\pm 5\%$. These values (2 cents, 0.5 dB) are chosen to be **subtle** – they won't make notes sound out of tune or mismatched in dynamics, but they will help avoid the exact same waveform summing and causing phasing or a robotic feel. This kind of micro-randomization is a lightweight alternative to round-robin samples, and is recommended by sound designers to add organic variation ²⁷ ²⁸. Note that Unity's `AudioSource.pitch` can go beyond normal range (e.g., 0.99 or 1.01 for a cent-level change is fine).

If a note has to be **stopped** (say, if we implement note cut-offs or a stop button), we can call `src.Stop()` or `src.Pause()`. Pausing and resuming might be useful if, for example, we implement a hold note that should stop when the tile is released, though in classical piano context this is more like releasing the sustain pedal.

Unity built-in vs Audio middleware: Unity's native audio system, as configured above, can meet the low-latency needs of this game. It's essentially using FMOD's low-level API under the hood ²⁹, but Unity does not expose all of FMOD's advanced features (e.g., no built-in reactive music system or sophisticated DSP graph in the free engine). **When might one consider adding FMOD or Wwise?** If down the line you need more advanced audio control – say complex layering of background tracks, real-time effects, or better cross-platform consistency – middleware could help. FMOD and Wwise both allow sample-accurate scheduling, support fancy effects (with the cost of their own CPU usage), and provide authoring tools for audio designers (for example, defining loops, transitions, or adaptive music behaviors without coding). They may also achieve slightly more consistent latency on Android by using newer APIs (FMOD recently can use AAudio on Android which might improve latency a bit). However, integrating them means adding an SDK (~50 MB library) and writing interface code.

For a first release, **sticking to Unity's built-in audio is perfectly reasonable and simplest** ⁵. Unity can trigger hundreds of one-shot sounds per second if optimized, and developers have successfully made rhythm games with it by focusing on the tips discussed (buffer size, preload, etc.) ⁴ ³⁰. If down the road you encounter insurmountable issues (e.g. timing drift on Android that you can't correct, or you want to use FMOD's timeline callback features to drive gameplay ⁶), you can consider migrating or hybrid use of FMOD. The **pros of middleware** include robust cross-platform audio handling and easier mixing of complex events, but the **cons** are added complexity and potential licensing costs (FMOD is free below a revenue cap; Wwise has tiered licenses). Given that Unity already uses an embedded FMOD, the baseline performance may not drastically differ unless you specifically use features like **timeline markers** to sync events (FMOD can send a callback on a music beat, for instance). In our case, since our "music" is just individual notes we trigger, we don't need those features yet. We will just ensure our Unity implementation is as tight as possible.

Touch input latency: Although not directly part of audio playback, it's worth noting: detect screen taps on **Touch Down** events, not the default Unity UI `onClick` (which fires on release). The Medium article warns against using Unity's UI Button for rhythm input because it triggers on button release by default ³¹. Instead, use `Input.touches` or a Physics Raycast to catch the instant the finger hits the tile. This ensures the visual feedback and audio (if any triggered by hits) are not delayed by an extra few frames of touch processing. Combining low audio latency with instant input makes the game feel snappy.

In short, **use Unity's audio with care**: smallest buffer, preload everything, pool your AudioSources, and schedule or trigger precisely. This will give you responsive note playback on mobile. Only if you hit a wall (like Android audio drift) should you consider a deeper solution (some devs have written native Android plugins to get exact playback timestamps ³², but that's a heavy route and likely unnecessary for our needs). Our approach will be to test on a range of devices and adjust as needed.

3. Classical Repertoire Considerations

Designing a note-triggered game for classical and other public-domain music entails both musical and legal considerations.

Pitch range of classical pieces: You should ensure your sample set and code can handle the full range of notes used by your target repertoire. Classical piano music from the Baroque and Classical eras (Bach, Mozart, early Beethoven) was often composed for smaller keyboards (5-6 octaves), so pieces rarely use the very bottom or top of a modern piano. For example, Bach's Two-Part Inventions stick to a fairly modest range – roughly from C3 up to maybe C6 or so – because harpsichords of his day had about 4-5 octaves. However, by the Romantic era (Chopin, Liszt, Debussy), composers fully exploited the 7+ octaves of the piano. **Liszt's virtuoso pieces and late romantic works can hit the extreme A0 or C8 keys** on an 88-key piano. A standard grand piano has 88 keys spanning A0 to C8 (A0 ~27.5 Hz, C8 ~4186 Hz). That is a **range of over 7 octaves**, covering ~80% of human hearing range ⁷. It's safe to assume any *advanced* repertoire could use any of those keys. As a baseline, you should at least cover the **A0-C8 range** in your sample set (even if by stretching the last few notes at the ends). If you plan to focus on mid-difficulty pieces initially, you might not need the extreme ends; for instance, many Chopin waltzes or Bach preludes don't go beyond 4 or 5 octaves span. But to be future-proof, include the full range. Running out of sample range (pressing a key that doesn't exist in your audio) would be a glaring issue.

Some data points: **Mozart's and Haydn's piano works** rarely go below F1 or above C7 because the pianos then had ~6 octave range. **Chopin's pieces** often go high (Chopin loved the very high tinkling registers in some nocturnes and etudes, around C7-C8) and occasionally quite low (bass notes in polonaises down to A0). **Rachmaninoff** and **Liszt** will definitely demand the lowest A0 and highest C8 at times. So if those are on your roadmap, you need full 88 coverage. In summary, for "cornerstone classical pieces" as you mentioned: a safe bet is **A0 through C8** inclusive. Using pitch-shifting, you could for example record up to, say, F#7 and pitch up to C8, and record down to C1 and pitch down to A0, if you wanted to trim a few. But ensure quality – extreme pitch shifts (like a perfect fifth shift) at the ends can sound bad. It might be worth having a sample of the lowest A or two, and the top C, even if everything else is more sparse.

Licensing public-domain music: The great news is that all compositions by Bach, Mozart, Chopin, etc., are firmly in the **public domain** (the authors died well over 70 years ago, which is the general copyright term in most jurisdictions) ⁸ ³³. This means you can use the notes of those compositions freely – you can perform them, record them, display the sheet music, etc., without paying royalties. However, there are a few **caveats** and tasks to ensure you remain in the clear:

- **Sheet music editions:** While the composition is PD, a specific edition of the sheet music (say, an urtext edition edited in 1990) might carry its own copyright (for things like fingerings, typesetting, or editorial additions). If you're obtaining scores to make MIDI files, use reputable PD sources such as **IMSLP/Petrucchi Library** or **Mutopia**. Those sites host scans or files of old editions that are out of copyright. Alternatively, use modern editions that explicitly allow free use

(some editors release to PD or under Creative Commons). If you transcribe by ear or type in notes from a known PD source, you're safe.

- **MIDI files or recordings:** MIDI sequences themselves can be copyrighted by whoever created them, even if they represent PD music ⁹. Many "free MIDI" websites exist for classical songs – some are truly PD, others are just user-uploaded and not clearly licensed. For example, **Classical Archives** offers thousands of MIDI files; they were historically a subscription service and may have usage restrictions. The best approach is either **create your own MIDI files** (e.g., by playing the piece on a MIDI keyboard or entering it in a notation program) or use MIDI files from a source that explicitly states they are PD or CC0. One such source is the **Mutopia Project**, which provides MusicXML/MIDI for many classical works that are typeset from PD sources. Another is **Musescore.com** community – often users share classical piece transcriptions (though check the license per file; many are free to download). Since you mentioned possibly converting scores, using **MusicXML** from IMSLP (some scores are available in MusicXML) or **MIDI from Mutopia** would be good. Always double-check: if a MIDI's metadata has an author name or if the site has terms, abide by them. In short, ensure "free MIDI" truly means free of copyright.
- **Recorded samples and performances:** If you were using actual recordings of performances, those would be copyrighted by the performer/record label (even if the piece is PD) ³⁴. But in our game, we are not using someone's performance; we are assembling our own performance via individual note samples. We do need to consider the **piano note samples** themselves: if you did not record them yourself, make sure they are from a source that allows redistribution (many university or hobbyist sample sets are Creative Commons). For example, the University of Iowa's **MIS Musical Instrument Samples** are public domain recordings of each piano key (decent for basic use). There's also a well-known free **Salamander Grand Piano** sample set (Creative Commons Attribution). Licensing of samples is separate from composition copyright – just be sure to have the rights to include the *audio* assets.
- **App Store / Google Play compliance:** Both Apple and Google simply require that you have rights to all content in your app. Since classical compositions are PD, you don't have to list anything special, but it can be polite to somewhere credit "Pieces by Frédéric Chopin are public domain". Apple in particular doesn't want any copyrighted music without proof of license, but PD should not trigger any issues. To be extra safe, you could include an in-game Credits or README that states "All musical works performed in-game are in the public domain (composed before 1927). Piano samples © [your name or source], used with permission." This covers all bases. As one developer noted, **recording it yourself or rendering it via MIDI means you owe nothing to anyone** ³⁵, whereas using someone's 2007 orchestral recording of Bach would be infringement. We're effectively doing the former – generating our own renderings.

Public-domain score & MIDI libraries: To drive your gameplay, you'll need a way to parse musical files (MIDI or MusicXML). Fortunately, there are solid **open-source C# libraries** available:

- **DryWetMIDI (by Melanchall):** A popular .NET library for reading/writing MIDI files. It's MIT-licensed (free for commercial use) ¹⁰. DryWetMIDI can open a `.mid` file and give you access to tracks, notes, tempos, etc., through a convenient API. It handles low-level details (delta times, ticks, etc.) and presents events in order. This is perfect for reading a MIDI of, say, *Für Elise* and extracting a list of note start times and pitches. Many Unity devs use this library (it's available as a Unity Asset or via NuGet). With it, you could parse a MIDI at startup and populate your game's note timeline.

- **MusicXml.Net (by vdaron on GitHub):** A C# library specifically for MusicXML files ¹¹. MusicXML is an XML-based sheet music interchange format, often exported by notation software. It contains rich info (notes, articulations, etc.). If you have MusicXML files (for example, from MuseScore or exported from Sibelius), this library can parse them into objects in C#. It's also open-source (looks like MIT license from what's typical). Example usage: `var score = MusicXmlParser.GetScore("path/to/file.xml");` which returns a Score object with a collection of Measures, Notes, etc. ¹¹. You can then iterate over those and build your note chart.
- **Sanford MIDI Toolkit or NAudio:** Historically, Sanford's MIDI Toolkit is an older open-source library for MIDI, and NAudio is a general audio library that can also parse MIDI. However, DryWetMIDI is more modern and well-maintained. Since DryWetMIDI covers our needs well (and has even been integrated into Unity by others), it's probably the go-to. NAudio is more for audio output and WAV processing – not needed here.
- **Others:** If you ever consider using Python for preprocessing, the `music21` library can parse MIDI/MusicXML and analyze them ³⁶, but that's outside of Unity. There are also **online converters** (like Tone.js MIDI-to-JSON ³⁷, VexFlow for rendering notation, etc.), but using a C# library internally is more efficient than relying on external conversion tools.

Using libraries in practice: The plan could be: maintain a collection of song files in either MIDI or MusicXML format (MIDI might be simpler). At build time or startup, use DryWetMIDI to parse the file into a list of notes (with their start times, pitches, and durations). Then convert that into your own **JSON chart format** (or ScriptableObject, or any data structure Unity can use at runtime). The JSON could look something like:

```
{
  "title": "Moonlight Sonata (1st mov)",
  "tempo": 55,
  "notes": [
    { "time": 0.0, "pitch": 57, "duration": 2.0, "volume": 100 },
    { "time": 0.5, "pitch": 60, "duration": 1.5, "volume": 100 },
    ...
  ]
}
```

Where `time` could be in seconds or beats, `pitch` is MIDI note number (57 = A3 for example), `duration` in seconds or beats (for judging long notes or maybe for sustaining visuals), and `volume` (MIDI velocity 0-127, if you decide to use that for dynamic intensity). You would also include any tempo changes or time signature if needed – for many classical pieces, tempo can fluctuate, but if you're strictly quantizing to a grid, you might decide to ignore subtle tempo rubato or encode it manually. If a piece has multiple sections with different BPM, your chart format should support a **tempo map** (e.g. an array of tempo changes by timestamp). MusicXML will have `<sound tempo="...">` markings or metronome marks you can read.

Tempo and rhythm alignment: Classical MIDI files often contain tempo variations (either expressive or simply to encode ritardando). For a game, you might want a simplified consistent tempo to make tile spacing uniform. You could either **average out the tempo** or pre-process the MIDI to quantize it. Alternatively, treat the exact timing as ground truth (which is more challenging for players if it isn't a steady beat). Many rhythm games stick to a fixed BPM per song section, even if the actual performance

would breathe. You'll have to decide whether to incorporate small tempo fluctuations or not. Since the game is about classical music performance, it might be interesting to allow slight rubato, but it could confuse players. A compromise: keep pieces at a fixed BPM for gameplay purposes. That means if a MIDI file has tempo changes, you might flatten them. Or ideally, find MIDI files that are mechanical (like from a notation program) rather than recorded from a human performance (which might have tempo drift).

Slicing into “time packages”: This phrase likely refers to breaking the chart data into chunks (perhaps measures or segments). One reason to do this is to **load or stream large songs progressively** rather than all at once. Another is for easier scheduling: you can deal with one segment at a time. For example, you could divide the piece by bars and package each bar's notes. Then, your game logic can load upcoming bars slightly ahead of time. This can help memory if you unload bars that have passed, though note data is usually so small that it's not a big memory issue (a 3-minute piece with thousands of notes in JSON is maybe a few hundred KB at most). However, some piano pieces (especially virtuosic ones) might have *thousands* of notes – consider Liszt's Campanella or flight-of-bumblebee runs. It's wise to be prepared for high note density.

From a performance standpoint, iterating through a huge list of notes each frame to see which ones to spawn can be inefficient. A common pattern: segment the notes by time, so you only consider a subset at any given moment. For instance, if you package notes by each beat or each half-second, you can quickly grab the notes for the next half-second when needed. Another approach is to use a **priority queue** or indexed structure keyed by time, so you pop notes as their time comes.

There's evidence that **Piano Tiles 2** and similar games stream in the note data and can slow down on older devices if the song is long or fast ³⁸. The Piano Tiles wiki notes that *Turkish March* (a fast, note-dense piece) uses more and more RAM as it speeds up, and older iPads lag or even run out of memory ³⁸. This suggests that their implementation might be loading notes (or creating tile objects) on the fly and not disposing quickly enough, causing a buildup. A robust approach for us: if a piece is extremely fast (say 20 notes/sec bursts), ensure we **destroy or reuse tile objects** promptly after they are off-screen, and possibly break the music data into smaller chunks so we never schedule too far ahead.

One strategy is to break the MIDI into **sections by bar**: e.g., create a list of bars, each containing notes relative to that bar's start. You can then load, say, 4 bars ahead and unload bars behind the playhead. Unity can handle thousands of small objects, but proactively managing them can prevent hiccups. If note spawning becomes a bottleneck, consider object pooling for the tiles similar to audio pooling.

Parsing libraries in Unity: Integrating DryWetMIDI or MusicXml.Net into Unity is straightforward (just add the .DLL or source code). Then you can create an Editor script to convert your files into a Unity-friendly format (like ScriptableObject or JSON text assets). Some developers run an offline conversion (e.g., use a Python script or a C# editor script) to bake all MIDI files into JSON files included in the build. This way, at runtime, you just do a quick JSON parse (which Unity can do via JsonUtility or a library like Newtonsoft JSON). The choice of JSON vs binary is up to you – JSON is human-readable for debugging and adequate in size for note data. Since the request specifically mentions JSON, we'll assume that route.

Finally, **music library recommendations**: The **Classical MIDI Archives** (classicalarchives.com) is a huge repository, but not all MIDIs are free for redistribution – read their terms. **Mutopia** and **IMSLP** provide either MIDI or MusicXML for many pieces (Mutopia has MIDI files contributed under CC0 or similar for each score). Also, the **MuseScore** online library has user-contributed scores – many are arrangements of classical pieces. You can often download those as MIDI or MusicXML. Just verify if they mention a license. Since these are exact classical pieces, usually the user cannot claim a new copyright on the *notes*

if they faithfully copied from PD sheet music (though technically their MIDI performance could be copyrighted, but many users upload under a share-alike license). To avoid any gray area, you could use those files as a reference but regenerate the MIDI via your own means.

In summary, **choose reliable PD sources, parse with robust libraries, and convert into a runtime format.** This will give you a solid foundation of note data to feed the game. Keep the chart data segmented and optimized for quick access, especially for long or dense compositions.

4. Data-Driven Chart Pipeline

A **data-driven approach** means the musical note information is externalized (not hard-coded) and can be processed systematically. We've touched on using MIDI/MusicXML libraries to extract the notes – now we focus on how to turn that into a game-ready chart and handle it during gameplay.

Converting MusicXML/MIDI to JSON chart: The conversion pipeline might look like this:

1. **Parse the source file:** Use DryWetMIDI or MusicXml.Net to read the original music file into an in-memory representation (list of notes with timings). For MIDI, DryWetMIDI gives you collections of `NoteOnEvent` / `NoteOffEvent` or a higher-level `Note` object which has properties like `StartTime` (in ticks) and `Length` (in ticks). You'll also get the file's division (ticks per beat) and any tempo change events. For MusicXML, you'd get measures, notes, and durations (often in terms of divisions too).
2. **Apply tempo map:** If the piece has varying tempo, decide how to handle it. You might convert everything to absolute time in seconds using the tempo indications. For example, if a MIDI is 120 BPM throughout, 1 beat = 0.5 seconds. If tempo changes, you accumulate time accordingly. DryWetMIDI can provide you a function to get time in milliseconds for each tick when given a `TempoMap`.
3. **Normalize or quantize (optional):** If you prefer to enforce a steady tempo or simplify rhythms, you could quantize note positions to the nearest 1/16 or so. But for authenticity, you may keep the exact values.
4. **Construct JSON data:** Iterate over all notes and output their properties. Often it's convenient to convert tick times to **beats or fractional beats** relative to a base BPM. Alternatively, convert directly to seconds if you plan to time everything by wall-clock time. Many rhythm games use beats internally because it's easier to speed up or slow down charts if needed. However, since classical pieces aren't likely to change tempo during gameplay (except intentionally as part of expression), using seconds is fine.

Include in the JSON any **global info**: initial tempo, perhaps time signature (if relevant for visual bar display), song name, etc. For tempo maps, you could include an array of {time, newTempo} entries if needed.

Example snippet:

```
{
  "title": "Turkish March",
  "composer": "W.A. Mozart",
```

```

    "bpm": 120,
    "beatUnit": 4,  // quarter-note beat
    "notes": [
      { "beat": 0.0, "pitch": 76, "duration": 0.5, "vel": 100 },
      { "beat": 0.5, "pitch": 78, "duration": 0.5, "vel": 100 },
      { "beat": 1.0, "pitch": 79, "duration": 1.0, "vel": 100 },
      ...
    ]
  }

```

Here, times are in beats at 120 BPM (so beat 1.0 = 0.5 sec after beat 0 in this case if BPM=120). Duration in beats can be used to draw hold notes if needed, or just for scoring (most piano tile games don't hold notes, they just count taps, but you might want to handle notes that sustain beyond their hit).

1. **Automation:** You can write an Editor script to batch convert all your `.mid` or `.musicxml` files into `.json` text assets. This way you can easily update charts by re-exporting the source and running the converter. There are also online or command-line converters: e.g. `mid2json` scripts or the **VisiPiano** tool for MIDI-to-JSON ³⁹, but integrating a library is cleaner and doesn't require leaving Unity.

Tempo maps and sync: Representing the tempo is important for gameplay sync. If you keep times in seconds in the JSON, then the `bpm` field is more for display (or for relative speed adjustments). If you keep times in beats, then `bpm` is needed to convert to real time. For a consistent visual scrolling speed, you probably will use a constant BPM (like each beat corresponds to a certain distance movement on screen). If a piece actually changes tempo, you have a design choice: reflect that in gameplay (tiles slow down or speed up accordingly), or cheat a bit and keep the scroll speed constant, which would desync actual music time unless you also change the playback speed – better to actually reflect tempo changes if they are large (like a *rallentando*). Minor expressive tempo fluctuations might be ignored for simplicity.

Packaging and slicing: After generating the full note list, consider splitting the data for runtime. One method: group notes by **measure** (since classical pieces are written in measures). You can include bar markers in JSON, or just compute measure boundaries using time signature info. Another method: group by fixed time window, like every 1 second or every beat.

For example, create an array `packages`: each containing notes for a span of, say, 2 beats or 1 bar. This could look like:

```

"packages": [
  { "startBeat": 0.0, "notes": [ ... notes up to beat 4.0 ... ] },
  { "startBeat": 4.0, "notes": [ ... notes from beat 4.0 to 8.0 ... ] },
  ...
]

```

During gameplay, you then load packages sequentially. If the song is long, you might not want to load the entire JSON at once (though JSON parsing a few hundred KB is very fast, so it's usually fine). You could also stream it from Resources or an asset bundle, but probably unnecessary.

The main performance concern is **iterating over upcoming notes**. You don't want each frame to scan a huge list for notes that fall into the next frame window. A simple approach: keep an index into the sorted note list, and advance it as time passes. This way, each note is processed once. This is efficient even for thousands of notes. So maybe you don't need to actually split the JSON, as long as you manage an index pointer. However, splitting might make it easier to handle dynamic loading/unloading if memory or organization is an issue.

Automating slicing: If you did want to automate slicing into time-based chunks, you could incorporate that into the conversion script: iterate through notes in time order and whenever a note passes a threshold (like 2-second mark or measure boundary), close the current package and start a new one. There aren't off-the-shelf tools specifically for "slicing into rhythm game packages" beyond these general programming techniques, since it's usually custom to the game's architecture.

Some resources from game devs: The **FMOD forum topic "Using frame look ahead to beat latency"** suggests preparing events and pausing/playing them to avoid latency in a rhythm game ⁴⁰ – not directly related to slicing, but indicates pre-preparing segments of music. And the **Unity forum "Impossible to do a rhythm game with Unity due to latency?"** basically concluded that decomposing tasks and scheduling ahead is the key ⁴¹. That implies splitting out events (like our note events) and handling them in advance.

In summary, the pipeline to get from classical sheet music to in-game tiles is:

- Obtain public-domain **score data** (MIDI/MusicXML).
- Use **parsing libraries** to extract structured note data.
- **Convert and normalize** that data into a custom JSON format (with tempo info, etc.).
- At runtime, **load the JSON** (or have it pre-baked as a Unity asset), iterate through the note events in time order to spawn tiles and schedule sounds.
- Optionally, **segment the data** to manage large volumes or facilitate level streaming.

This approach ensures that adding a new song is as simple as dropping in a new MIDI file and running the converter, rather than coding new behavior. It also makes it easier to update charts (say, if you want to simplify a piece for an easier difficulty, you could edit the MIDI and reconvert).

By making the game rely on this data, you can quickly expand the repertoire or adjust note timing without touching game logic – a hallmark of a robust data-driven design.

5. Mobile Performance & Memory Optimization

Mobile games must be optimized to run smoothly within limited CPU, memory, and battery constraints – especially a fast-paced rhythm game where missed frames or delayed audio are noticeable. Let's consider performance expectations and known bottlenecks, using piano-tiles clones and our design as context.

Target performance metrics: We aim for **60 FPS** gameplay with stable frame times. Audio should play with no audible lag or dropouts, even during rapid note sequences (potentially **200+ notes per second** in extreme cases, though typical classical pieces average far less). Battery usage should be reasonable – rhythm games tend to be always-on and can heat up a device if not optimized. We can't avoid using CPU/GPU (we have constant motion and audio), but we can avoid waste.

CPU considerations (audio): Mixing many simultaneous sounds is actually not as heavy as it sounds – mixing 32 voices of short clips is usually fine on modern phone CPUs, since the audio thread runs at a lower rate (e.g. 48000 Hz, doing vectorized mixing). The risk is more about *scheduling* overhead if we trigger an excessive number of AudioSource.Play calls in a short time. Unity's audio system can handle quite a few – but triggering 200 distinct sounds in one frame is obviously extreme. In classical music, 200 notes *per second* might occur as 20 notes each 0.1s apart, not all at the exact same instant. So the distribution matters. A rapid chromatic run might fire, say, 10 notes in one frame (if they're within ~16ms). That's still okay, but it's something to test on low-end devices. Using the audio pool avoids alloc/dealloc overhead for those bursts.

One potential issue is **Garbage Collection (GC)** if you allocate frequently. Our design avoids that: we reuse objects and our note data is loaded once (JSON parsing allocates a bit, but after that we just traverse lists).

Memory (audio): As covered, decompressing all samples might take, say, ~10-20 MB of RAM. This is acceptable on a 2019 phone (typically 2-4 GB RAM). Unity's default memory for audio is limited by the voice count (32 by default) and any buffers. If we raise voice count to 64 or so, the audio system might allocate slightly more memory for mixing buffers. But more significant is **the memory used by tile objects and textures** (if any), which we should keep eye on. The piano tile games usually have minimal graphics (black/white tiles), so low memory footprint there.

The **Piano Tiles Wiki** notes that certain songs cause lag on older devices due to memory – specifically, *Turkish March* on an iPad 2 or iPhone 4S could slow down, likely because of insufficient RAM for the song's needs ⁴². They even mention that song “uses more and more RAM than any other when it gets sped up” ³⁸. This suggests maybe their implementation loads tiles progressively and doesn't unload fast enough, causing buildup. On devices with 512 MB (iPad 2), that became an issue. They also mention those songs won't lag if you have a device with ~4 GB RAM ⁴³, which is way more than needed – but basically, memory was the bottleneck on old devices.

Our plan to mitigate that: ensure that for any given song, we do not accumulate unbounded game objects or data. Use object pooling for tiles as well (reuse tile prefabs instead of instantiating endlessly), and destroy or reuse them once they pass the screen. That will keep memory steady.

CPU/GPU (graphics): The moving tiles and touch input will likely be the main load on the **game thread** and GPU. While not explicitly asked, note that drawing 60 FPS is important, as visual lag or stutter will throw off rhythm. Techniques to optimize this include using Unity's **UI canvas** or a simple 2D sprite approach for tiles, and avoiding expensive per-frame operations (like complex physics or too many canvases). Many piano tile clones use simple rect transforms and perhaps a single canvas. If using Unity's UI system, watch out for canvas rebuild overhead if lots of elements change. Using a dedicated canvas for moving tiles might help.

Audio spikes on spikes of notes: The question specifically asks strategies to avoid “AudioSource pool exhausted” on spikes. We already addressed ensuring the pool is sufficiently large. Additionally, plan for **polyphony capping**. If somehow 100 notes try to play at once (maybe a huge cluster chord in a modern piece, or an arrangement quirk), playing all might not even be audible distinctly. It could be better to cap. Unity will cap at the voice limit automatically. So setting voice limit to, say, 64 means note 65 won't play (or will virtualize). If we know no musical piece should legitimately have >64 simultaneous distinct notes (which is a safe assumption – even mashing the sustain pedal on a piano, physical strings damp out and you effectively won't have all 88 ringing loudly at equal volume), we can accept 64 as an upper bound. Using **voice priorities**, we can ensure if the limit is exceeded, the quietest notes drop first ¹³. In practice, maybe it's okay to let Unity handle it. But to be safe, we could implement our own logic: e.g.,

if pool is full, don't play new note or stop the oldest. Because our pool is round-robin, if we naively call `Play()` on a source that's still playing, Unity will *immediately cut off* the old sound and start the new one on that source. That can be a form of voice stealing (last one in replaces one that wasn't finished). This might be okay if it's rare, but it could cause audible cut-offs (imagine a sustain pedal case where a long note is truncated by a new note). A smarter voice-stealing could fade out the longest note over 0.1s rather than cut it. Such nuance might be beyond scope, but it's a thought for polish.

Benchmark expectations: It's hard to find published CPU/RAM benchmarks of piano tile games, but anecdotal evidence suggests they are not very heavy – after all, they run on modest phones and focus on simplicity. The heavy parts are the frequency of touches and updates at high speed. Some reports on forums mention Unity's audio can glitch if too many events pile up in one frame (like playing many sounds in a tight loop); the solution is usually to stagger them or ensure the audio thread isn't starved. We've already mitigated decode overhead by preloading. If we still had issues, one could consider combining note samples into one audio stream – but that defeats the purpose of dynamic triggering and is more like just playing a pre-recorded track.

Battery usage: Unity games often get a reputation for battery drain if not optimized ⁴⁴. Constant 60 FPS means high CPU/GPU usage. We can mitigate battery usage by enabling **VSync** to 60 (to avoid rendering more frames than needed), using **Application.targetFrameRate = 60** explicitly, and possibly providing an option for 30 FPS if users want battery saving (though 30 FPS might harm game feel). Also avoid unnecessary **Update()** calls – use one manager Update to handle all logic rather than dozens on each tile object, for example. These micro-optimizations reduce CPU overhead and thus battery. On the audio side, there's not much to do except keep the audio thread work minimal (which we have by decompressing audio and not using heavy DSP).

Case study – “Slow Device”: As per the Piano Tiles Wiki's *Slow Device* entry, they describe how some players exploited lag to make the game easier (slowing down means you effectively get more time to react) ⁴⁵. To prevent any such issues or cheating, we want our game to run at consistent speed on all devices. That might involve using **FixedUpdate** tied to music time or ensuring `deltaTime` doesn't spike. If the device is too slow, maybe dropping graphical detail or capping polyphony (to avoid audio stutter) would be needed. We might consider adding a toggle for “Lower latency vs Better battery” which toggles DSP Buffer size or frame rate, but likely not necessary if we manage well.

Audio thread starvation: Unity's warning for too many streams is relevant: “Some users have reported high CPU use on mobile when streaming multiple AudioClips at once” ¹⁹. We are not streaming (we avoid that for notes), so we're good. But if we had a background track, we'd stream that and the rest decompressed. Keep it that way to not overload decoding.

Recap of strategies to avoid performance issues:

- Use **object pooling** for both audio and tile GameObjects to avoid runtime allocations.
- Keep the audio **voice limit** high enough (e.g. 64) to handle bursts, but not so high that the CPU tries mixing an absurd number of voices. Unity suggests if you need >80 voices, reconsider your design ²⁵ – we will stay well under that.
- Exploit Unity's built-in **voice culling** by setting priorities if needed ¹³. For example, if a soft sustain is ringing and a new loud note comes in, Unity will naturally favor the loud one if voices are maxed ¹³.
- **Cap polyphony in design:** If a particular arrangement yields an unrealistic pile-up of notes (like if you convert a piece with sustain pedal where every note sustains fully), you might limit sustain length or voices. In a game context, one could decide not to sustain notes infinitely, or even

implement a soft polyphony limit (like older MIDI synths did, where they'd stop the oldest note when limit reached).

- **Profiling:** Unity's Profiler and Audio Profiler will be invaluable ⁴⁶ ⁴⁷. You can watch how many AudioSources are playing, CPU usage, etc., during a song. This will guide where to optimize further.

To ensure we never exhaust the audio pool unexpectedly, one could include a runtime check: e.g., if all sources are playing when a new note arrives, log a warning (or implement a fallback like steal the oldest). Testing extreme scenarios (like a dense MIDI file of a cluster chord glissando) will help validate the system.

Finally, consider **threading**: Unity's audio runs on a separate thread for mixing. But all our scheduling and Play() calls happen on the main thread. If we have to schedule dozens of notes at once, spreading them over a few frames or using `InvokeRepeating` might avoid blocking a single frame. But realistically, calling Play() even 30 times in one frame is not that heavy (each is a fairly lightweight C++ call to FMOD). We just should avoid doing huge work (like JSON parsing) during gameplay; load all that upfront.

In conclusion, a combination of **preloading, pooling, capping, and profiling** will keep the game performing well. Other developers' experiences show that careful settings can eliminate latency and that Unity can handle rhythm game demands when tuned ⁴ ⁴¹. We'll adopt those best practices to ensure even a rapid piece with 200 notes/sec doesn't either drop frames or audio on a typical mid-range phone.

6. Enhancement Techniques for Realism & Variety

To elevate the audio experience, we can incorporate several techniques that add depth and reduce repetitiveness, without straining performance.

Reverb (environmental ambience): Piano music typically sounds best with some room reverb – a completely dry sampled piano can feel unnatural or clinical. We have two main options: **runtime reverb (convolution or algorithmic)** or **baked reverb tails** in the samples.

- *Convolution Reverb (Real Impulse Responses):* This gives the most realistic sound (you can sample a concert hall's reverb and apply it), but it is **extremely expensive on CPU for mobile**. Game audio experts strongly caution against convolution reverb on mobile VR/AR or any performance-intensive app ⁴⁸. One comment succinctly says: *"Convolution reverbs are ridiculously expensive... Keep well away from them."* ⁴⁸ Another mentions a project where just 2-3 convolution reverbs ate ~80% of the audio CPU and caused buffer underruns, especially with long 4s IR files ⁴⁹. Clearly, real-time convo reverb is not advisable for our use case.
- *Algorithmic Reverb:* This refers to synthetic reverbs (like Unity's built-in **Audio Reverb Filter** or simple plate reverbs). These are **much cheaper** and you can tweak them to approximate a room sound ⁵⁰. Unity's built-in reverb filter can add a sense of space at low CPU cost. The quality won't match a true convolution of a cathedral IR, but with tuning (room size, decay, etc.) it can be acceptable. The Reddit thread suggests algorithmic reverbs "most likely work" for real-time if tweaked ⁵⁰. So a viable approach is: attach a Reverb Filter to the AudioListener or use an AudioManager send for reverb, and set a preset (Unity provides presets like Hall, Auditorium, etc.). Monitor CPU usage – it should be okay if not too many sources send into it heavily.
- *Baked reverb tails:* This is an interesting approach specifically hinted in the GameAudio thread: *"render the reverbs as separate files with a quad reverb and position that around the player"* ⁵⁰ and *"baked convolution tail to reduce length of realtime convolution"* ⁵⁰ ⁵¹. In our context, this could

mean: take each piano sample and offline convolve it with a desired IR (hall reverb), then **store only the tail portion** as a separate clip (or extended sample). One could then play that tail sample quietly after the main note, or even loop a low-level ambient tail. However, doing this for every note multiplies assets and complicates playback (you'd need two AudioSource per note: one for the dry and one for the tail send). A simpler variant: pick a single IR that suits all notes (like a consistent hall), convolve a representative note, and use that as a **global reverb send**. But reverb is linear and can be summed – so perhaps better: just use an algorithmic reverb globally.

Given performance vs benefit, a recommended approach is: **use a lightweight global reverb** to get some space. Perhaps Unity's "Room" or "Hall" preset at low mix level (like 20-30% wet) so that it's subtle. This avoids needing per-note processing and costs maybe a few % CPU. If even that is problematic, a trick is to bake a **static ambience**: e.g., have a stereo track of a piano reverb tail (like the diffuse part) playing in loop, which the notes duck into. This might be overkill. Start simple: no reverb or a tiny bit, and see if the sound is acceptable. Remember, we can always add reverb offline in the sample creation: if our note samples were recorded in a room, they already have natural decay. Some free piano samples are recorded dry, some semi-wet. If they are dry, the game will sound a bit like a piano in a vacuum without any reverb.

The consensus from experts: **avoid convolution on mobile** ⁴⁸, use **algorithmic if needed** ⁵², or **bake the tail** (which essentially is like having pre-rendered reverb in the samples) ⁵⁰. So we will follow that advice.

Velocity layering and dynamic EQ: If you include actual velocity layers (multiple samples per note for different intensities), that's the best for authenticity – e.g., soft presses have a mellow tone, loud presses have a bright, percussive tone. If not, we can simulate some aspects. As noted earlier, **piano timbre changes with velocity – not just volume** ⁵³. A key observation is that loud strikes produce more high-frequency content (the hammer hits harder, generating more overtones). Also the **attack of the note** is sharper. To mimic this with one sample per note, consider applying a **filter or EQ based on velocity**. For instance, for a note intended to be loud, you could boost a high shelf EQ a couple dB to add brightness, or conversely, for a very soft note, low-pass filter it slightly to make it more muted. Some digital pianos do this: they link velocity to a filter cutoff to emulate how timbre brightens with intensity ⁵⁴.

Unity doesn't have a built-in per-AudioSource EQ that's easily tweaked on the fly (there is an Audio LowPass/HighPass filter component that can be added to sources). One could attach a **LowPassFilter** to each AudioSource and set its cutoff frequency based on velocity (higher cutoff for higher velocity to allow more highs through). For example, velocity 127 -> cutoff 22kHz (no filter), velocity 20 -> cutoff maybe 5kHz to muffle it. This would add some CPU but it's a simple filter per source, likely fine for moderate polyphony. Alternatively, route all notes into an AudioMixer group and use an EQ effect there, modulating via snapshots or exposed parameters. However, since multiple notes with different velocities play simultaneously, a single group EQ won't differentiate them. So per-source filter is better if going this route.

Another element is **volume curve vs velocity** – simply scaling volume linearly with velocity might not feel right; pianos have a more exponential response (double velocity is much more than double perceived loudness). Unity's default if you just use the velocity value as volume might be okay, but consider tuning the curve (e.g., use a quadratic scale).

If implementing velocity at all, you also have to consider how to obtain the velocity values: if using a MIDI, you have them. If using MusicXML, you might not have explicit velocities unless encoded as dynamics (f, p markings). You could then map, say, 'f' to velocity 100, 'p' to 40, etc. It might be simpler

initially to ignore velocity (treat all notes equally) for gameplay, but that loses musicality. Perhaps as an enhancement, you implement a system where if a piece has dynamics, the game reflects them in audio (and maybe even in scoring if you care about expression). But most rhythm games focus on hitting notes in time, not how hard.

In summary, a practical step: **Use at least 2 velocity layers if possible** (maybe a “forte” sample set and a “piano” sample set). If not, apply a **dynamic filter**: e.g., attach a `LowPassFilter` on each `AudioSource`; when playing a note, set `cutoffFrequency = baseCutoff + (velocity/127)* (maxCutoff - baseCutoff)`. Base might be 3000 Hz for `vel=1`, `maxCutoff` 22000 Hz for `vel=127`, for instance. This approach was suggested in synthesis contexts ⁵⁴ and can add a subtle but effective variation in tone with minimal overhead.

Dynamic EQ could also involve slight adjustments to mid frequencies to simulate **pedal sympathetic resonance** (when sustain pedal is down, piano has a richer sound). That is advanced – some digital pianos simulate the harmonic resonance of un-damped strings. Implementing that fully is complex. But a simple approximation: if sustain pedal is on (if you allow it in some songs), maybe add a slight reverb or a slight boost to certain frequencies to make it sound more “ringy”.

Micro-randomization (humanization): We already introduced one aspect: random slight pitch and volume changes per note to avoid robotic repetition. This is especially helpful if you have only one sample per note and no round-robin – hitting the same note 5 times quickly can otherwise phase or sound too static. By detuning each a tiny bit (like ± 1 – 2 cents), you ensure no two hits interfere coherently. Similarly, a ± 0.5 dB volume difference and maybe a tiny stereo pan randomization (like $\pm 5\%$ L/R) can add stereo width. Many game engines or audio middlewares have a built-in “random pitch” property for one-shots – Unity doesn’t directly in the `AudioSource`, but you can easily script it as shown. This technique is widely used in shooters (for gunshots) and any repeated sounds ²⁷. It’s basically free in terms of performance (just computing a random number). The main caution is to keep the range small enough that it’s not noticeable as wrong notes. Our ± 2 cent suggestion is imperceptible musically but effective.

Tackling repetition artifacts: In sections of music with repeated notes (e.g., a pedal point or a trill), using round-robin samples if available is ideal. If not, the randomization covers some of it. Another trick: slightly vary the **attack envelope**. We can’t do that easily per `AudioSource` in Unity (we’d need to edit the sample or apply an envelope). If it’s crucial, one could prepare two versions of a note sample: one with a tiny offset or slightly different attack, and alternate them. That’s essentially manual round-robin.

Stereo positioning: Note that many piano sample sets are recorded in stereo from the pianist’s perspective or an audience perspective (high notes to the right, low to left). If yours is, maintain that to give a natural stereo image. If your samples are mono, you can simulate stereo by slight panning: e.g., pan low notes a bit to the left, high notes to the right. It adds clarity when many notes play. Unity can handle that via `AudioSource.panStereo`. This is not exactly “micro-randomization” but a deliberate pan mapping. It’s subtle and won’t cost much performance.

Expressiveness vs gameplay: In a classical performance, tempo rubato, dynamics, and pedaling all contribute to expression. In a game, especially a tile tap game, too much variation can confuse the player (they expect steady timing and clear feedback). We should carefully decide which musical nuances to include. For instance, adding a **sustain pedal effect** (longer note decays and resonance) would make it sound more authentic, but if the gameplay doesn’t involve a pedal mechanic, it might muddy the sound for the player’s taps. You might opt to simulate only partial pedal (shorter decay than

real but longer than staccato) or none at all. However, pieces like Chopin's rely on pedal for the legato sound – playing them completely dry would lose character. One approach: allow sustained decays but maybe avoid piling up too loudly.

Dynamic difficulty or adaptive sound: Though not asked, an enhancement could be adapting audio based on how the player performs (like if they miss notes, maybe drop those notes from the mix or play a muted sound to indicate a mistake). Some rhythm games do that – missing a note silences it or plays a wrong sound. This requires separating the “backing” from the “note” – in our case, there is no separate backing track, so if they miss a note, maybe that note just doesn't sound (which itself is the punishment – the music is incomplete). That inherently is handled: if they don't trigger the note, it won't play. But if we auto-play everything, that's different. Just something to note when designing scoring/feedback.

To wrap up: **Enhance audio carefully:** - Use **small random variations** in pitch/vol (cheap, effective). - Possibly implement **velocity sensitivity** through either multi-samples or filters to add realism for soft vs loud passages (the player might not control velocity, but the piece's notation does). - **Reverb:** provide a sense of space without heavy cost – likely a global send reverb with an algorithmic solution or minor convolution only if pre-baked. GameAudio pros explicitly advise to “*stay away from convolution on mobile*” due to its cost ⁴⁸. If ultimate piano realism is desired later, maybe as an option on high-end devices, one could toggle a convolution reverb IR on/off, but it's low priority. - If multiple microphones or EQ: possibly, if your piano samples have multiple mic positions (some libraries do), stick to one for simplicity. Or you could simulate lid position or EQ changes as a fun feature (not necessary). - The **end result** should be that the piano sounds rich and not fatiguing even after long play. Tiny variations and reverberation help achieve that, preventing the audio from feeling too synthetic.

7. Roadmap & Risk Mitigation

Finally, we outline a development **roadmap** from prototype to release, highlighting key milestones and potential risks at each stage.

Implementation Roadmap

Phase 1: Prototype (Core Mechanics)

- **Set up basic Unity project:** Import audio sample clips (initially maybe a reduced set, e.g. one octave, for quick testing). Configure Project Settings (Best Latency DSP, etc.). Prepare a simple scene. - **Implement AudioSource pool:** Write the pooling system for note playback (as in section 2). Verify that pressing a key (or a debug UI button) instantly plays the correct note sound with minimal delay. - **Prototype input and tile spawn:** Create a simple mechanism to spawn a falling tile and register a hit (this can be very rudimentary initially, even just tapping anywhere to trigger next note). - **Load a test MIDI chart:** Hard-code or load a small sequence of notes (e.g., a C major scale or a short melody) and use it to drive tile spawns and note plays. This will test the timing system. - **Verify timing and latency:** On a device (Android and iPhone if possible), measure the sync between visual and audio. Adjust as needed (tweak buffer, ensure Decompress on Load is set for test clip). At this stage, you might use a simple metronome click sound with known offset to fine-tune.

Phase 2: Data Pipeline & Full Song Playback

- **Integrate MIDI/MusicXML parser:** Import DryWetMIDI or MusicXml.Net. Write an Editor converter that turns a MIDI file into your JSON (or directly into a ScriptableObject containing the note array). - **Create a sample song chart:** Use a well-known simple piece (e.g., *Bach Minuet in G* or *Twinkle Twinkle* melody) as a guinea pig. Convert it and load the data in-game. - **Implement chart timing logic:** Use the parsed note times to schedule tile spawns and audio. Likely maintain a song clock that advances

(e.g., based on `AudioSettings.dspTime` if you start a scheduled track, or based on `Time.time` if you manually progress with no backing track). Since we are generating the music via notes, you might drive the timing off an internal metronome. - **Visual feedback loop:** Ensure that tiles appear in sync with the audio notes. This might require calibrating a **lead time** (spawn the tile X ms before the audio so that it reaches hit line exactly when audio plays). This is the classic “audio-visual offset” issue. You might start with X equal to average latency and adjust. - **Basic scoring/judgment:** Implement a simple hit detection (if tap within some tolerance of note time, it’s a hit). This is not audio-related, but important for the game logic to test full loop. - **Memory and performance check:** Play through the full test song, watch the Unity Profiler for spikes. Check that the audio pool size is sufficient (no warnings about virtualizing voices unless intended), and that CPU usage for audio stays reasonable (Unity Profiler’s audio graph).

Phase 3: Expand Content & Features

- **Build the full sample set:** Record or gather all piano note samples needed (e.g., at least every 2nd or 3rd note across 88, or all 88 if decided). Integrate them, assign them to their note pitches in a manager so that given a MIDI note number we know which AudioClip to play and at what pitch transpose (if any).
- **Parsing multiple songs:** Convert a batch of classical pieces into charts. Set up a song select and menu (so you can test switching pieces).
- **Implement velocity & pedal (if planned):** At this stage, incorporate dynamic filtering for velocities or multiple sample layers, and ensure the parser captures dynamics markings or MIDI velocities. Also, if you want sustain pedal behavior, implement it (notes should not stop when key is released if sustain is on; you can simulate by extending note durations or by not stopping the AudioSource when a sustain message is on).
- **Add polish to input:** Possibly integrate multi-touch (so users can hit chords) and optimize touch handling to be instantaneous (use `Input.Touch` or low-level events).
- **Graphical improvements:** Make the tiles and UI look nice (won’t detail here, but ensure it doesn’t break performance).

Phase 4: Optimization & Testing

- **Profile on actual devices:** Use mid-range Android (2019 model with say Snapdragon 665 or equivalent) and an older one if possible, as well as a mid-range iPhone. Test demanding pieces (fast, dense notes). Look for: - Audio latency or hiccups. If some devices still have delay, consider adding a **calibration setting** for the player (common in rhythm games) – e.g., an offset they can adjust. Or find if certain devices need `AudioSettings.GetDSPBufferSize` adjustments or even native audio plugins.
- CPU usage spikes. If a certain passage causes audio crackle, see if it’s due to too many voices – then possibly limit simultaneous voices slightly or reduce decode overhead (maybe use ADPCM on those samples).
- Memory usage. Ensure that loading a large song doesn’t blow memory. If needed, use Unity’s `Resources.UnloadUnusedAssets` after a song to clear clip data if it’s not reused.
- **Battery/thermal testing:** Run a longer session and see if the phone heats up drastically. If so, consider frame rate limiting on menus, pausing audio when not needed, etc.
- **Edge cases:** What if a song has a very long sustained note? Does it ring out properly without abruptly cutting off? (We might need to allow looping on samples or use the sustain pedal logic.) What if two notes are extremely close (like 64th notes)? Does the game register and audio play distinctly? Tweak pool or judgement windows accordingly.

Phase 5: Additional Enhancements

- **Audio enhancements:** Introduce the subtle reverb to improve sound (test that it doesn’t throw off latency – Unity’s reverb should be stable). Adjust randomization ranges for best sound (listen critically with headphones).
- **Dynamic music feedback:** If you plan any feedback for misses (e.g., maybe mute missed notes or play a gentle error sound), implement that. This might involve checking if the player tapped a note; if not, perhaps lower the volume of that note’s AudioSource or prevent it from playing. This is tricky since the music is driving the gameplay, not following it. Another approach: you always play the track, and missing a note only affects score (the original Piano Tiles games do this – the music

plays regardless, you just fail if you miss). - **Finalize content:** Make sure all included songs are correctly licensed (they should be PD; ensure any MIDI arrangement used was legal – perhaps credit sources like “MIDI sequences from Mutopia, used under CC0”). Also confirm that the sample library license allows inclusion in a game (if you recorded it or if it’s CC0, fine; if it’s CC-BY, be sure to attribute; if it’s a purchased library, check the EULA). - **User settings:** Provide options such as volume control, maybe latency calibration, possibly graphics quality toggle for low-end devices (like simpler backgrounds to save GPU).

Phase 6: Soft Launch & QA

- **Beta test:** Release to a small group or region. Collect performance data (maybe integrate Unity Analytics or at least get anecdotal reports). See if any device models consistently have problems (e.g., some Androids might still have ~100ms audio latency due to hardware – you might document that or see if a native plugin like Oboe (Android high-performance audio) could help in future). - **Bug fix:** Fix any crashes or timing issues discovered. For instance, there could be edge cases where the JSON parsing fails for a certain piece (maybe unusual time signature). - **Prepare release build:** Strip out any debug code, ensure the app meets store size limits (if it’s close to, say, 150 MB on Google Play, consider using AAB and asset packs for additional songs).

Release: At this point, you should have a robust engine. For initial release, maybe include a curated set of classical pieces, and ensure everything runs smoothly on target devices.

Major Risks & Mitigations

Despite careful planning, certain **risks** could impact development or release. Here are key risks and how to mitigate them:

- **Audio Latency Variability:** As discussed, different devices (especially Android) have varying inherent audio latency (some as high as 200 ms) ²². **Mitigation:** Provide an in-game calibration tool (a visual flash and sound click where user taps in sync to measure offset, then adjust note timing accordingly). Also, document recommended device specs (e.g., note that older Android versions before O low-latency features might have issues). For serious competitive play, targeting iOS or newer Androids might be ideal, but we can’t exclude users – just handle it by calibration.
- **Performance on Low-End Devices:** There’s a risk that older phones (pre-2015) might lag or overheat. **Mitigation:** Include graphical quality settings (lower frame rate or simpler effects on low-end). Ensure the game can drop to 30 FPS gracefully if needed (maybe automatically if it detects consistent slow frames). Use pooling to avoid GC hitches. Test on at least one low-end device (e.g., an old Android with 2GB RAM). If it struggles, consider limiting certain features (maybe cap maximum notes/sec dynamically if needed, or disable reverb on low-end). Also, implementing a frame skip in extreme cases (like slowing down the game if device can’t keep up) – though that’s not ideal for a rhythm game.
- **Application Size Limits:** Including many high-quality samples and multiple songs can bloat the app. Google Play’s base APK limit is 150MB (expansion files can go beyond, but that’s another complexity). Apple App Store allows up to ~200MB for cellular download, beyond that users need Wi-Fi. **Mitigation:** Compress audio aggressively (Vorbis quality tuning). Ditch any samples or songs that aren’t needed initially. Possibly offer additional songs as downloadable content (AssetBundles or addressables) so the core app stays lean. For example, ship with a few songs and let user download more from a server – but that requires infrastructure. Alternatively, if size is an issue, consider using **MPEG-4 AAC** for the clips on iOS since those can be hardware-

decoded (Unity doesn't automatically, but you could play them via AVAudioSession externally if desperate for size). However, likely with Ogg Vorbis at ~q4 quality, you can get a full piano ~5-10 MB and each MIDI file is tiny, so the songs themselves negligible. So the biggest chunk might be the Unity engine overhead itself. Keep an eye on final build size.

- **Licensing Pitfalls:** While the compositions are PD, mistakes here could include inadvertently using a MIDI sequence that someone copyrighted or using a sample library not cleared for games. **Mitigation:** Double-check licenses: use only **public-domain or CC0** MIDI/MusicXML sources when possible. If using any Creative Commons content (like a sample library under CC-BY), include proper attribution in credits (this satisfies license and avoids store rejection if someone reports it). Ensure no trademarked material in game name or such either. Since you plan to add "royalty-free modern music" later, be careful: "royalty-free" doesn't always mean free of copyright – often it means you licensed it. For modern music, you'll either need to use truly license-free tracks (like those under CC or original compositions) or go through proper licensing (which can be expensive and complicated). Perhaps stick to **modern music that is also copyright-free** (e.g., maybe indie artists who release under CC). That's a future consideration.
- **Audio Engine Integration Risk:** If later adding FMOD/Wwise, the risk is that integration could introduce bugs or latency if not done perfectly. It's an optional path; the mitigation is to thoroughly test the built-in approach first and only integrate middleware if absolutely needed (which might not be, as we expect).
- **User Experience Risks:** If the timing is off or device differences cause frustration, players might give up. Mitigation: robust calibration, perhaps an auto-calibration on first launch (some games play a sound and use microphone to measure round-trip latency – but that's complex and often not needed). Also, provide practice mode or slower speeds for very hard pieces so players don't blame the game for their misses.
- **Frequent Updates to Song List:** Each new song adds content. The pipeline we set up makes it easy, but ensure to retest performance on any *particularly* dense new song. Some modern compositions (if you ever include, say, *Flight of the Bumblebee* or something) push limits; test those specifically.

By following this roadmap and being mindful of risks, we can proceed from a working prototype to a polished release systematically. Each step builds on the prior, ensuring that by the time of release, we've addressed technical hurdles such as latency, performance on various devices, and licensing clarity.

Finally, maintain thorough **documentation**: note any measured latency on test devices, settings used, etc., so that if issues arise in user hands, you can quickly relate to known profiles. And include a way (perhaps a hidden debug menu or a logging feature) to get feedback from users if something's off (for example, if a particular phone model has problems, having a way to output its audio config might help debug).

Actionable Bullet-Point Summary (for Unity implementation):

- **Audio Assets:** Import piano note samples and compress as **Ogg Vorbis** (quality ~50%) for small size. Mark all note AudioClips as **Decompress on Load** for low-latency playback ⁴. If CPU decoding becomes an issue, switch some clips to **ADPCM** compression (lower CPU) ³.
- **Audio Settings:** Set **Project Settings > Audio > DSP Buffer Size = Best Latency** (small buffer) to minimize output lag ⁴. Increase **Virtual and Real Voice Limits** to ~64 (from default 32) to allow high polyphony without voice cuts ²⁴ ¹².

- **AudioSource Pool:** Pre-create a pool of ~50 **AudioSource** objects at startup to reuse for note playback. Avoid `PlayOneShot` for notes (since we need per-note pitch control); instead, assign clip and use `AudioSource.Play()`. Implement a simple **round-robin** or **free-list** system to cycle through sources. This prevents runtime allocation and ensures bursts of notes won't exhaust available sources.
- **Low-Latency Playback:** Use `AudioSource.Play()` or `PlayScheduled()` a few milliseconds before the exact needed time to counter frame scheduling jitter. Ensure the **AudioListener** and sources are all at default positions (3D spatialization off or 2D) unless you intentionally pan notes. Turn off any **DSP effects** on AudioSources (no default spatialization, etc., to keep them instantaneous).
- **Music Data Pipeline:** Use **DryWetMIDI** (MIT license) to read MIDI files at edit-time and convert them into a lightweight JSON format containing note timings (beats or seconds) and pitches ¹⁰. Load this JSON TextAsset at runtime to get the note sequence. This allows easy addition of new songs without code changes.
- **Chart Timing and Spawning:** Create a **song clock** that progresses according to the chart's tempo. Spawn note tile GameObjects a fixed lead-time before their hit time (e.g., spawn 2 seconds early so they fall into position). On spawn, also schedule/trigger the note's audio via the AudioSource pool. Use the same timing source for animation and audio to keep them tightly in sync.
- **Touch Input Response:** Use **OnTouchDown** (or `Input.touches` in Update checking phase == Began) to detect taps immediately. Avoid Unity UI Buttons for gameplay input (they fire on release by default) ³¹. Map touch positions to the corresponding tile/note and provide instant feedback (lighting up tile, etc.). This ensures minimal input latency impact.
- **Performance Profiling:** Frequently test on target devices using Unity Profiler's **Audio** module. Check that the **Audio DSP load** remains low (e.g., under 10%) even during dense passages, and that **no "voices culled" warnings** appear (indicating voice limit is sufficient) ⁵⁵. Also monitor **Memory** to confirm audio clips are preloaded (they'll show in memory as decompressed size).
- **Voice Management:** Utilize Unity's **AudioSource Priority** if needed – set higher priority on important voices (melody) and lower on accompaniment, so if voice count is exceeded, accompaniment drops out first ¹³. If all notes equal priority, Unity will cull by quietest sound – which generally aligns with musical importance (so it's okay).
- **Reverb & Sound Enhancements:** For initial release, keep reverb subtle to avoid latency. If using, add a **Global AudioMixer Reverb** at 20-30% wet mix for ambience instead of per-source convolution ^{48 50}. Implement **micro-pitch randomization** (± 1 -2 cents) and volume variance (± 0.5 dB) on each note trigger to avoid robotic repetition ²⁷. If possible, integrate at least **two velocity layers** for samples (soft and loud); if not, use Unity's **Audio LowPassFilter** on sources to darken soft notes and open up for loud notes, simulating velocity response ⁵⁶.
- **Sustain Pedal Simulation:** If the music demands pedal, implement logic to **hold AudioSources** instead of stopping them on note-off while pedal is active. You can tie this to MIDI sustain controller messages. Use a gentle fade-out or Unity's **AudioSource.Release()** (not an actual API, but conceptually) when pedal is lifted to avoid abrupt cuts. This adds realism for pieces like Chopin where pedal is essential.
- **Memory Management:** Keep total audio memory in check – for example, if using ~88 samples * ~0.5MB each uncompressed = ~44MB, consider compressing or loading selectively. Unity's **LoadInBackground** can be enabled on clips to avoid hiccup on scene load ^{57 58}. After a song, if memory is an issue, you can **UnloadUnusedAssets()** to clear audio clips from RAM if you unload their references.
- **Calibration & Options:** Include a **manual audio timing calibration** offset in settings. This lets players adjust if they perceive a desync. Also allow toggling of effects like reverb or v-sync (battery saver) to accommodate different devices.
- **Testing and QA:** Test on a variety of devices (Android with high latency, iOS which is usually low latency, different hardware). Use a **high-speed test song** (e.g., a MIDI with 16th notes at 200 BPM) to stress-test the system. Ensure no audible glitches (if found, adjust pool size or investigate if DSP buffer needs to be even smaller – on Android, Unity might limit how low it goes). If necessary, consider using

native audio for Android (Oboe or OpenSL via a plugin) only if Unity cannot achieve acceptable latency on a critical device (this is a last resort given complexity).

By following these action items, you'll apply best practices from game audio research directly into your Unity project, laying a solid foundation for a responsive and robust piano rhythm game.

Sources: 1 2 3 4 5 6 8 9 10 11 48 50 27

1 GitHub - ukushu/UnityPianoGame: Easy way to create Unity piano with a great piano sound
<https://github.com/ukushu/UnityPianoGame>

2 Soundfont piano .zip | Musical Artifacts
<https://musical-artifacts.com/artifacts?formats=zip&tags=soundfont%2Cpiano>

3 12 13 16 17 18 19 20 23 24 25 46 47 55 57 58 10 Unity Audio Optimisation Tips - Game Dev Beginner
<https://gamedevbeginner.com/unity-audio-optimisation-tips/>

4 21 22 26 31 32 41 Rhythm game with Unity3D: achieve latency free sync (android and other platforms) | by Thibaut Dumont | Medium
<https://medium.com/@thibautdumont/rhythm-game-with-unity3d-achieve-latency-free-sync-android-and-other-platforms-c05fa8e2718b>

5 6 Rythm game and latency - Unity - FMOD Forums
<https://qa.fmod.com/t/rythm-game-and-latency/20973>

7 54 Synthesizing Pianos
<https://www.soundonsound.com/techniques/synthesizing-pianos>

8 9 33 34 35 legal - Can I use popular classical music as my game's soundtrack? - Game Development Stack Exchange
<https://gamedev.stackexchange.com/questions/23496/can-i-use-popular-classical-music-as-my-games-soundtrack>

10 MIT license - melanchall/drywetmidi - GitHub
<https://github.com/melanchall/drywetmidi/blob/develop/LICENSE>

11 GitHub - vdaron/MusicXml.Net: Quick C# parser for MusicXML
<https://github.com/vdaron/MusicXml.Net>

14 48 49 50 51 52 Reverb efficiency : r/GameAudio
https://www.reddit.com/r/GameAudio/comments/jf6j1b/reverb_efficiency/

15 Grand Piano Sound Pack - Realistic, high quality, real-world sampling, full 88 keys, ogg,wav,mp3 files | Audio Sound FX | Unity Asset Store
<https://assetstore.unity.com/packages/audio/sound-fx/grand-piano-sound-pack-realistic-high-quality-real-world-samplin-180587?srltid=AfmBOorXDKtWKOW3r5jy1muOpOtL4XR7WY0arRS1Q1Jm-m5qpOzGLWWN>

27 28 Sharing a really basic but useful tip: If there's a repetitive sound in your game, try putting a random pitch on it! : r/Unity3D
https://www.reddit.com/r/Unity3D/comments/18ycc02/sharing_a_really_basic_but_useful_tip_if_theres_a/

29 Unity - Manual: Audio in Web
<https://docs.unity3d.com/6000.1/Documentation/Manual/webgl-audio.html>

30 Low latency for music based game - Unity Discussions
<https://discussions.unity.com/t/low-latency-for-music-based-game/618630>

36 Installing MusicXML Readers and File Formats (1) — music21 ...

https://www.music21.org/music21/docs/usersGuide/usersGuide_08_installingMusicXML.html

37 Tonejs Midi

<https://tonejs.github.io/Midi/>

38 42 43 45 Slow Device | Piano Tiles Wiki | Fandom

https://piano-tiles.fandom.com/wiki/Slow_Device

39 VisiPiano 2.0 MIDI to JSON Converter

<https://www.visipiano.com/midi-to-json-converter/>

40 Using frame look ahead to beat latency - FMOD Forums

<https://qa.fmod.com/t/using-frame-look-ahead-to-beat-latency/14808>

44 Is "Made with Unity" synonymous with "destroys your battery"? - Reddit

https://www.reddit.com/r/AndroidGaming/comments/8dw75d/is_made_with_unity_synonymous_with_destroys_your/

53 Does velocity matter? : r/edmproduction - Reddit

https://www.reddit.com/r/edmproduction/comments/yq70vx/does_velocity_matter/

56 Understanding Velocity Curves - Piano World Forum

https://forum.pianoworld.com/ubbthreads.php/topics/2243079/Understanding_Velocity_Curves.html